Fiche de lecture – Projet 9 : Deep Hedging

Module Finance des Marchés et ERP – Groupe 9
Membres : Marouane, Aymane, Sara, Meryem

1. Formules clés

Dynamique du sous-jacent (GBM, marché simulé).

$$S_{k+1} = S_k \exp\Big[(\mu - \tfrac{\sigma^2}{2})\Delta t + \sigma\sqrt{\Delta t}\, Z_k\Big], \qquad Z_k \sim \mathcal{N}(0,1),\ \Delta t = T/n$$

Le prix évolue de façon aléatoire log-normale ; c'est le « marché » sur lequel on entraîne et on teste les stratégies.

Payoff de l'option vendue.

$$Z = \max(S_T - K,\, 0) \quad \text{(call européen)}$$

C'est ce que le vendeur doit payer à l'échéance : c'est la perte qu'on cherche à compenser avec la couverture.

Delta de Black-Scholes.

$$\Delta_k = N(d_1), \qquad d_1 = \frac{\ln(S_k/K) + (r + \sigma^2/2)(T - t_k)}{\sigma\sqrt{T - t_k}}$$

Le Delta est le nombre d'actions à détenir à la date t_k pour neutraliser la sensibilité de l'option au prix du sous-jacent ; c'est la stratégie classique de référence.

P&L de la couverture avec coûts de transaction.

$$\text{P\&L} = -Z + \sum_{k=0}^{n-1} \delta_k\,(S_{k+1} - S_k) - \sum_k c\,|\delta_k - \delta_{k-1}|\,S_k, \qquad \delta_{-1} = 0$$

On additionne le gain (ou la perte) du portefeuille d'actions, on retire le payoff dû et les coûts proportionnels c payés à chaque réajustement de la position.

Réseau de neurones.

$$\delta_k = f_\theta\big(\ln(S_k/K),\ T - t_k,\ \delta_{k-1}\big)$$

Le réseau reçoit l'état du marché (prix relatif, temps restant, position précédente) et renvoie directement la quantité à détenir, sans formule analytique.

Objectif d'entraînement.

$$\min_\theta\ \text{Var(P\&L)} \qquad (\text{ou } \text{CVaR}_\alpha,\ \text{ou } \tfrac{1}{\lambda}\ln\mathbb{E}[e^{-\lambda\,\text{P\&L}}])$$

On cherche les poids θ qui rendent le résultat final le plus stable possible : un P&L peu dispersé autour de zéro signifie une bonne couverture.

Hypothèses. Marché simulé (GBM ou Heston) : le réseau n'est valable que dans la dynamique d'entraînement ; coûts c connus ; fréquence de rééquilibrage fixe ; trajectoires d'entraînement indépendantes, avec séparation train/test pour mesurer la généralisation.

2. Lien avec le TP 0 (module Odoo `cours_derives`)

On ne repart pas de zéro : on étend le module existant dans le même dossier d'addons.

- Nouveau modèle `cours.simulation.couverture` avec les champs : méthode de couverture (sélection « Classique Delta » / « IA »), paramètres de marché (S_0, K, T, r, σ), coût de transaction c, nombre de trajectoires ; et des champs résultats stockés (moyenne, écart-type, VaR du P&L).
- Bouton « Lancer la simulation » (méthode `action_lancer_simulation()`) qui appelle le code Python du dossier `src/` (module interne ou petit service API) et enregistre les résultats ; vue formulaire + vue graphique comparant les deux méthodes.
- Le réseau est entraîné *hors d'Odoo* (notebook / PyTorch) et sauvegardé dans un fichier `.pt` ; Odoo le charge et fait seulement de l'**inférence** (pas de ré-entraînement, trop lent).
- On réutilise la fonction de prix et le Delta de Black-Scholes du Groupe 1 (`cours.option`) : s'accorder sur les noms de champs et les conventions (unités de T, annualisation).
- Fichiers à mettre à jour : `models/`, `views/`, `security/ir.model.access.csv`, `__manifest__.py`.

3. Algorithme en Python

1. **Simuler les prix** : générer M trajectoires GBM vectorisées avec NumPy/PyTorch (ex. 100 000 pour l'entraînement, 20 000 pour le test, $n = 30$ à 252 pas), graine fixée pour la reproductibilité.
2. **Calculer le P&L** : pour chaque date k, calculer δ_k (Delta de BS ou sortie du réseau), mettre à jour le gain sur les actions, retirer les coûts de transaction, puis soustraire le payoff à l'échéance.
3. **Entraîner le réseau** : MLP à 2 couches cachées (64 neurones, ReLU), optimiseur Adam (taux 10^{-3}), perte = variance du P&L, nouvelles trajectoires à chaque epoch, arrêt anticipé sur trajectoires de validation.
4. **Comparer les stratégies** : Delta de BS, aucune couverture / aléatoire, réseau ; indicateurs : moyenne, écart-type, quantile 5 %, coût de transaction total, histogrammes superposés.
5. **Analyser et intégrer** : tracer $\delta(S)$ du réseau contre le Delta de BS, étudier l'effet de c (le réseau apprend à moins rééquilibrer quand les coûts augmentent), sauvegarder le modèle en `.pt` et l'appeler depuis Odoo.